

Opening UPPAAL

The UPPAAL GUI requires a Java Runtime Environment installed. They recommend Java version ≥ 17 (see: <https://uppaal.org/downloads/>). All branches include a copy of UPPAAL that was downloaded directly from their website.

Linux: Open the folder `uppaal-5.0.0-linux64`, right click the file `uppaal`, and select “Run as a Program.” This will launch the UPPAAL application. Alternatively, UPPAAL can be launched from the commandline by running `java -jar uppaal.jar` from within the `uppaal-5.0.0-linux64` folder.

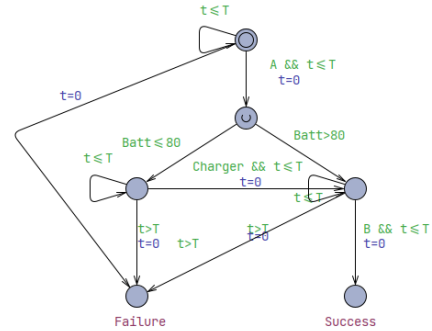
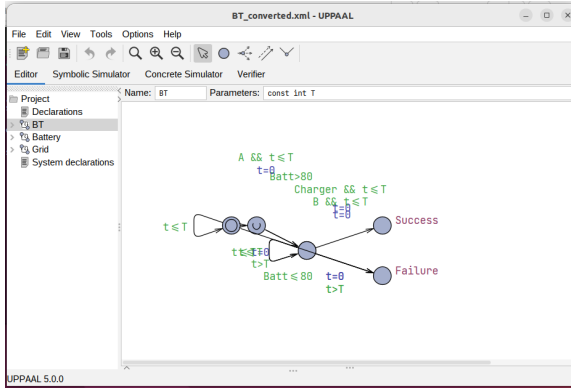
Windows: Open the folder `uppaal-5.0.0-win64` and double click the file `uppaal.exe`. This will launch the UPPAAL application. Alternatively, UPPAAL can be launched from the commandline by running `java -jar uppaal.jar` from within the `uppaal-5.0.0-win64` folder.

MacOS: The included file `UPPAAL-5.0.0.app` is a MacOS application which can be launched directly.

UPPAAL requires a license in order to run. I have included my academic license in the rebuttal and in the instructions in the Google drive. I have not included it in the repo for obvious reasons. If the copy is not already activated, copy the license code and enter it when prompted.

After Running the Script

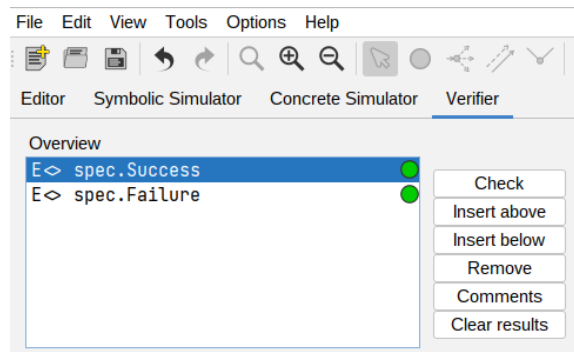
Running the script will produce a new file `BT_converted.xml` containing the BT equivalent automaton based on Fig. 2 from the paper. Also included in the new file is the automata modeling the battery dynamics and the grid world. Launch UPPAAL and select from the menu: **File** $\rightarrow$ **Open System...** Navigate to `~/hsc25-rp/bt2ta/` and select the file `BT_converted.xml`. Open the file. You will be greeted with the following screen.



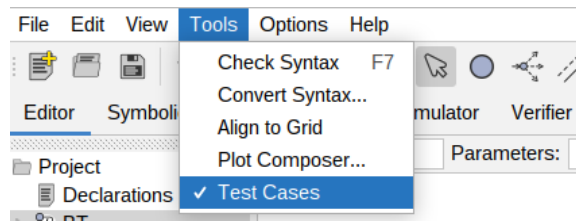
Feel free to navigate through the templates. The converted BT initially looks messy as the implementation does not try to organize the composed automaton. The locations and edges can be dragged around to make it more presentable. *However, this **does not affect** the program functionally.*

Verification with UPPAAL

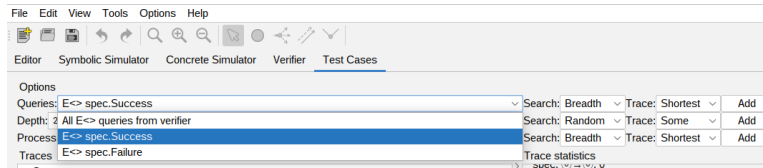
Two queries are automatically included under the **Verifier** tab: `E<> spec.Success` and `E<> spec.Failure`.



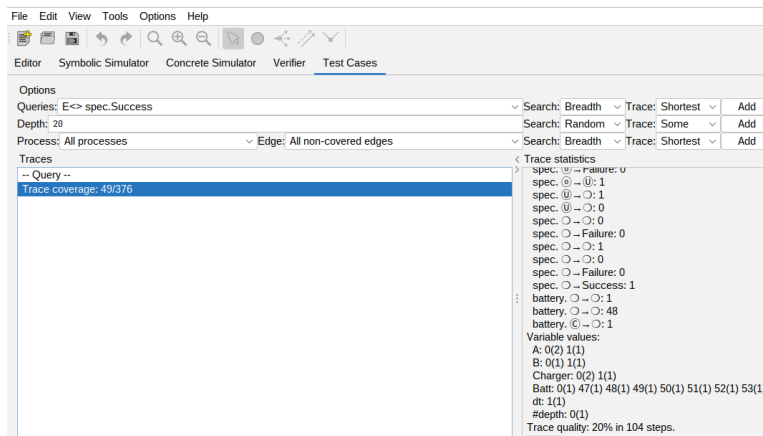
These queries demonstrate that the BT is capable of reaching the accepting state **Success** by successfully navigating the grid world. It is also capable of reaching the accepting state **Failure** by remaining in a single location as time elapses. To show the successful trace open the **Test Cases** tab. This is not shown by default and can be accessed by selecting **Tools** → **Test Cases** from the menu bar.



The **Test Cases** tab should now be available. Click on it to open the **Test Cases** window. Select from the **Queries** drop down menu **E<> spec.Success** and press **Add** to generate the trace.

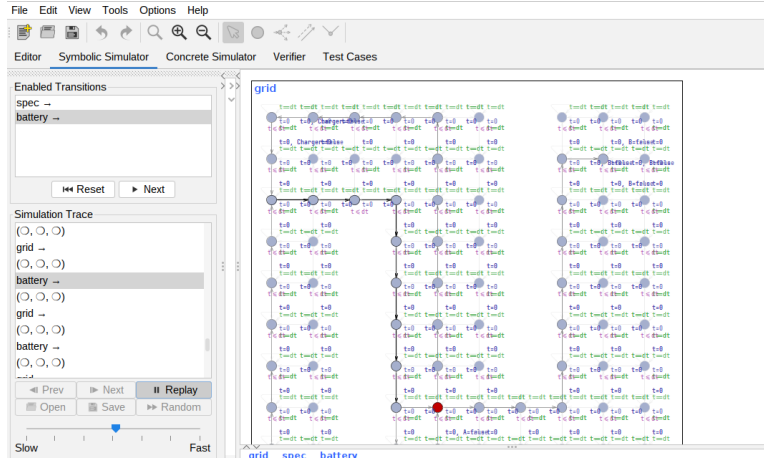


To simulate the trace, double click on **Trace Coverage** to select the trace. Then navigate to the **Symbolic Simulator** window.



The selected trace should now be displayed in the **Symbolic Simulator** window. The reviewer can parse through the **Simulation Trace** sub-window and replay the trace from the beginning. This trace represents the path navigated by the mobile robot in order to satisfy the BT specification. To make

the trace more visible, select the menu option **View → Coverage → Heatmap**. This will highlight the visited locations and fired edges.



The run is initialized with the battery charge set to 75%. The reviewer can edit the battery's initial charge by selecting the **Editor** window, **System declarations** tab and editing the argument in the **Battery()** declaration to any value between 75 and 100. To re-simulate the trace, the reviewer must save the model, navigate to the **Test Cases** window and re-add the specification `E<> spec.Success`.

Expected Behavior

- The robot will always find a satisfying trace for $T \geq 20$.
- If the initial battery charge is in $[97, 100]$, the robot will navigate to goal A and directly to goal B without stopping at the charger station.
- If the initial battery charge is ≤ 96 , the robot will navigate to goal A , then to the charging station, then finally to goal B .